

HCLTECH × ET MASTERCLASS — PROJECT 1

Quarterly Results Research Assistant

A retrieval-augmented research tool over four quarters of Apple Inc. earnings filings

Harshit Kumar Sharma

harshitkrsharma07@gmail.com

Repository: github.com/Harsh1t-S/finance-rag

Submitted 12 August 2026

1. The problem

GPT-4o has never read Apple's June 2026 quarter. Ask it for that quarter's revenue and it does one of two things: refuses, or produces a confident, plausible, wrong number. For an analyst drafting a client note, the second outcome is considerably worse than the first.

This system removes the model's need to know anything. Four quarterly earnings press releases are indexed; a question is answered **only** from the passages retrieved out of those documents, and every figure carries the document name and page number it came from, so it can be checked against the filing before it reaches a client.

The corpus

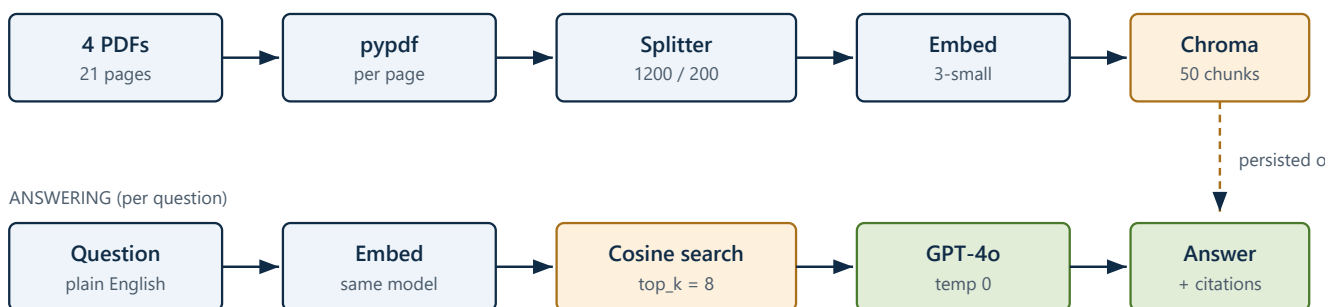
Apple files each quarter's results as Exhibit 99.1 to a Form 8-K. All four documents are public on SEC EDGAR.

Quarter	Period ended	Filing
Q4 FY2025	27 Sep 2025	8-K filed 30 Oct 2025, accession 0000320193-25-000077
Q1 FY2026	27 Dec 2025	8-K filed 29 Jan 2026, accession 0000320193-26-000005
Q2 FY2026	28 Mar 2026	8-K filed 30 Apr 2026, accession 0000320193-26-000011
Q3 FY2026	27 Jun 2026	8-K filed 30 Jul 2026, accession 0000320193-26-000018

Why the press release and not the audited statements. Apple also publishes a "Consolidated Financial Statements" PDF each quarter, which was the obvious first choice and was rejected. It contains tables and nothing else. The brief asks what management said about the demand outlook, and what risks are disclosed — neither question has an answer anywhere in a pure statements PDF. Exhibit 99.1 carries the same financial tables **plus** the CEO and CFO commentary, the dividend declaration with its record date, and the forward-looking-statement risk language. Every one of the ten test questions has a home in it.

2. Architecture

INDEXING (once, offline)



The model never sees a whole document.
Only the 8 retrieved chunks reach the context window — which is what makes an ungrounded answer detectable.

The indexing path runs once and persists to disk. The answering path runs per question and touches the LLM exactly once, with a context window containing nothing but the eight retrieved chunks.

Component	Choice
PDF reading	<code>pypdf</code> , page by page
Chunking	<code>RecursiveCharacterTextSplitter</code> , 1200 / 200
Embeddings	OpenAI <code>text-embedding-3-small</code>
Vector store	ChromaDB, persisted to <code>chroma_db/</code>
Answering model	GPT-4o, temperature 0
Interface	Streamlit
Backend (bonus)	FastAPI + Uvicorn
Secrets	<code>.env</code> via <code>python-dotenv</code> , gitignored

3. The pipeline, in code

3.1 Page-level extraction

Text is pulled out one page at a time and the page number travels with it. This is the decision that makes citation possible at all — concatenating the document first and splitting afterwards would let a chunk straddle a page boundary, and there would be no honest number to cite.

```
def load_pdf_pages(path: Path) -> list[tuple[int, str]]:
    reader = PdfReader(str(path))
    pages: list[tuple[int, str]] = []
    for i, page in enumerate(reader.pages, start=1):
        text = page.extract_text() or ""
        if text.strip():
            pages.append((i, text))
    if not pages:
        raise ValueError(
            f"No selectable text found in {path.name}. It is probably a scanned "
            f"image. OCR is out of scope for this assignment."
        )
    return pages
```

The explicit failure matters. A scanned PDF silently yields empty strings, which would index cleanly as zero chunks and produce a system that answers every question with "not available" for no visible reason.

3.2 Chunking, with deterministic identity

Each page is split independently. Every chunk is keyed by a SHA-256 of its filename, page and text, so re-indexing the same file **upserts over itself** instead of appending a second copy.

```

for page_no, page_text in pages:
    for piece in splitter.split_text(page_text):
        piece = piece.strip()
        if len(piece) < 40: # drop headers/footers that split off alone
            continue
        digest = hashlib.sha256(
            f"{filename}|{page_no}|{piece}".encode("utf-8")
        ).hexdigest()[:24]
        documents.append(piece)
        metadatas.append({"source": filename, "page": page_no, "chars": len(piece)})
        ids.append(digest)

```

Without deterministic IDs, pressing **Index** twice leaves 100 chunks where there should be 50. Nothing errors; retrieval quietly degrades as duplicate chunks crowd each other out of the top-k. The guarantee rests on the ID being a pure function of its inputs, which section 5 measures directly.

3.3 The refusal instruction

The single most important line in the project is the first rule of the system prompt. Without it, GPT-4o will invent a figure that reads exactly like a real one.

```

SYSTEM_PROMPT = """\
You are an equity research assistant at an investment advisory firm.

Answer ONLY from the context provided below. The context is extracted from
quarterly earnings press releases filed by the company with the SEC.

Rules:
1. If the context does not contain the answer, reply exactly: "That information
   is not available in the uploaded documents." Do not guess, do not use general
   knowledge, and do not fill a gap with a figure you happen to know. An analyst
   will act on this and an invented number is worse than no answer.
2. Always state which fiscal quarter each figure belongs to. "Revenue was $109.4
   billion" is useless without the quarter attached.
3. Cite the document name and page number inline for each fact you assert,
   in the form [Document name, p. N].
"""

```

Rule 2 exists because a figure without its quarter is not an answer to any question in this domain. Rule 3 is what makes the whole system auditable.

3.4 Labelled context

Chunks are labelled before they reach the model, so the citation it produces is grounded in something structural rather than in the model's guess about where the text came from.

```

def _format_context(documents: list[str], metadatas: list[dict]) -> str:
    blocks = []
    for i, (doc, meta) in enumerate(zip(documents, metadatas), start=1):
        blocks.append(
            f"[Chunk {i}] Source: {meta['source']} | Page: {meta['page']}\n{doc}"
        )
    return "\n\n---\n\n".join(blocks)

```

4. Design decisions

Chunk size 1200, overlap 200

The corpus is table-dominated, and at 800 characters the tables split mid-table, detaching row labels from figures.

A chunk containing 45,781 41,198 without the Americas label above it is worse than no chunk at all, because the model will still answer from it — fluently, and wrongly. 1200 is the top of the range the brief permits and keeps each table intact. The 200-character overlap keeps a heading such as "Segment Operating Performance" attached to the table beneath it.

top_k = 8, corrected from 6 during testing

Three of the ten questions compare figures **across quarters**, so they need chunks from three or four different PDFs simultaneously, and each quarter's net income sits in that quarter's own condensed statements table.

This value was wrong at first, and the way it was wrong is the point of the whole project. Measured on "compare net profit across all the quarters you loaded":

top_k	Quarters reached
6	3 of 4 — Q1 FY2026 missing
8	4 of 4

At 6 the system answered: "The highest net profit was in Q3 FY2026, with a net income of \$29,789 million."

Fluent, correctly cited, and **wrong** — the true answer is Q1 FY2026 at \$42,097M, from a filing retrieval never reached.

This is the most dangerous failure mode in the system, because the output looks entirely normal — well formatted, correctly cited, and quietly missing a quarter. Reading the answer will not reveal it. That is why the check is mechanical: `run_test_questions.py` prints how many distinct source documents were actually retrieved for every multi-quarter question, and it is what caught this.

```
MULTI_QUARTER = {"Comparison", "Year on year", "Trend"}

files_hit = {src["file"] for src in result["sources"]}
if kind in MULTI_QUARTER:
    verdict = (
        f"{len(files_hit)} of 4 quarters retrieved"
        if len(files_hit) > 1
        else "ONE DOCUMENT ONLY - check top_k"
    )
```

Temperature 0

Two analysts asking the same question of the same filing must get the same answer. Sampling variety is a feature for prose and a defect for financial reporting.

yfinance was deliberately omitted

The brief offers it as an optional extra. It pulls live market data that cannot be verified against the indexed documents. Mixing an unverifiable live number into an answer whose entire value proposition is "every figure traces to a cited page" would undermine the system.

5. Verification

Chunking was measured directly by running the real `load_pdf_pages` and `chunk_pages` functions over `data/`. The rest was measured by running the full pipeline end to end and reading what came back.

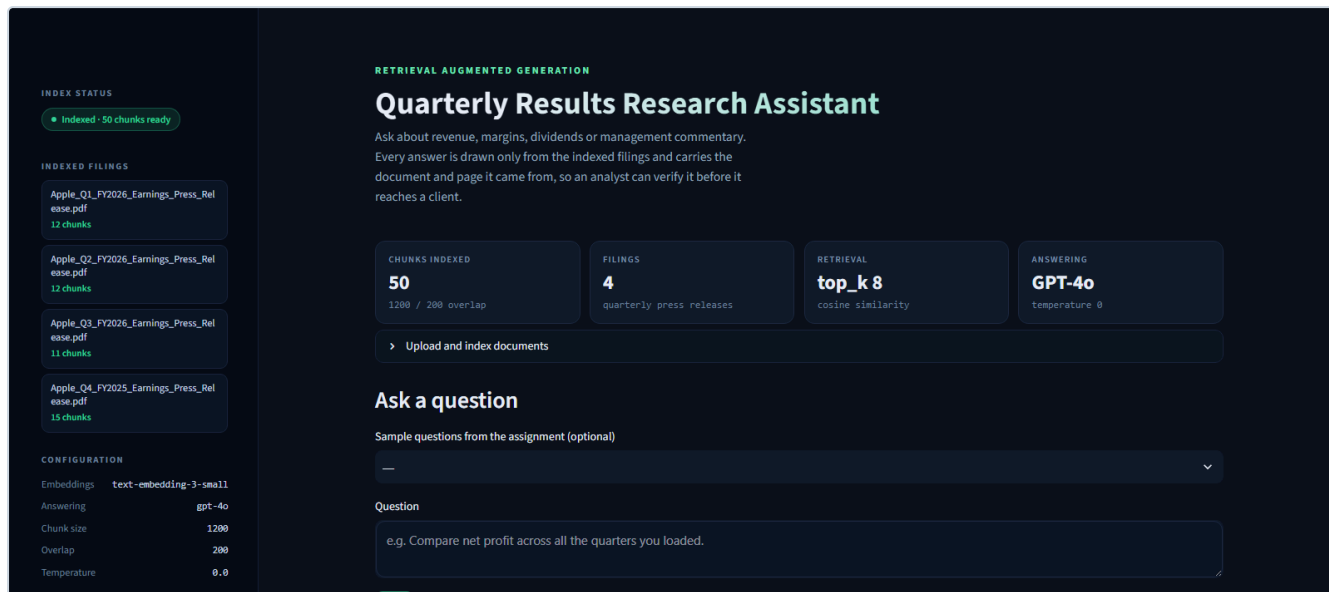
Check	Method	Result
Chunk count	<code>python ingest.py</code>	50 chunks from 4 files / 21 pages
Per-file split	page-level chunking	12 / 12 / 11 / 15
Largest chunk	<code>max(len(chunk))</code>	1200 chars — at the cap, none over
ID uniqueness	<code>len(set(ids)) == len(ids)</code>	50 of 50 unique , no collisions
Embedding + storage	<code>text-embedding-3-small</code> → Chroma	50 vectors persisted to <code>chroma_db/</code>
Ten test questions	<code>run_test_questions.py</code>	all answered, output in <code>docs/test_answers.md</code>
Retrieval spread	multi-quarter questions	4 of 4 quarters on all three
Trap question	Q10	refused verbatim: <i>"That information is not available in the uploaded documents."</i>

Idempotent re-ingest follows structurally from the ID row rather than from a separate experiment: the chunk ID is a pure function of `filename|page|text`, so an unchanged file produces byte-identical IDs and Chroma's `upsert` overwrites in place. The 50-of-50 uniqueness result is what makes that argument sound.

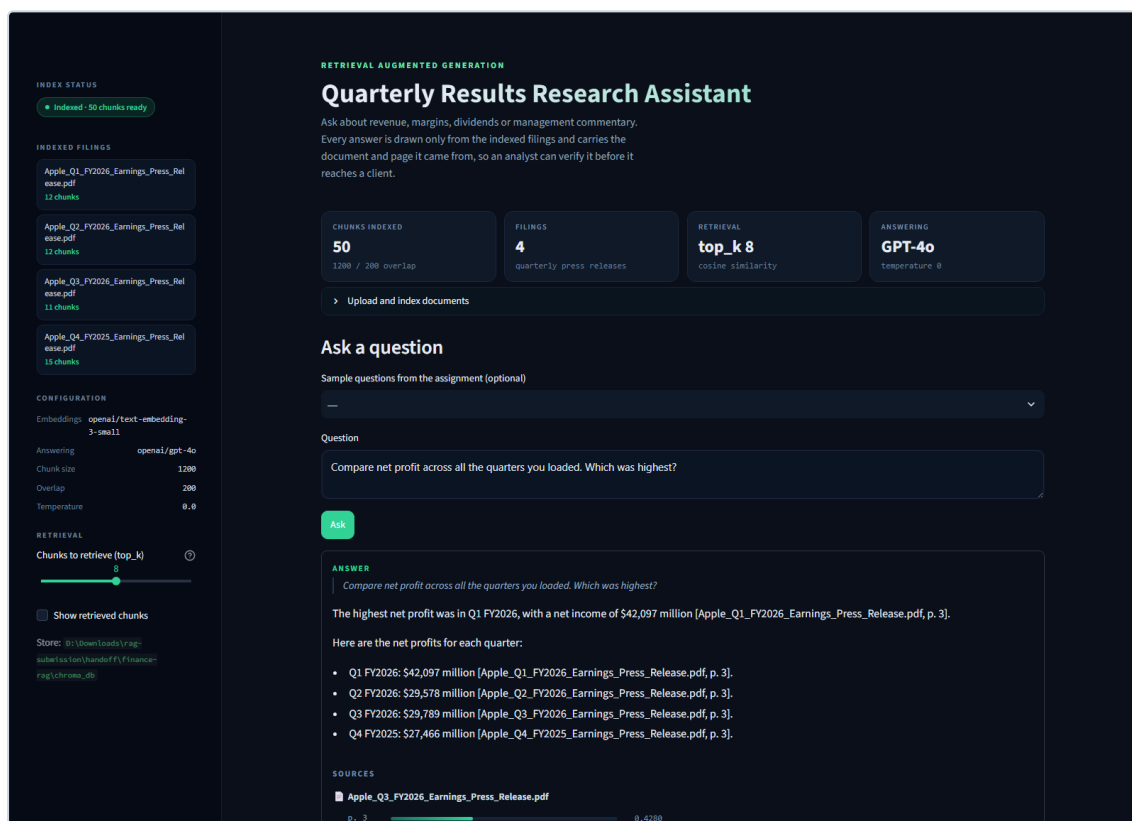
Where the numbers come from. Every figure above is from an actual run, not an estimate. The answers the system produced are committed to `docs/test_answers.md` with the sources, similarity scores and retrieval spread behind each one, so any claim here can be checked against the run that produced it.

Correction to an earlier draft. Prior notes recorded the largest chunk as 1198 characters. Re-measured, the finance corpus peaks at exactly **1200**; 1198 is the supply-chain figure. The substantive claim — that no table is split — holds either way, but the number is stated here as measured.

6. The working system



50 chunks indexed and persisted across the four filings, with models and chunk settings shown in the sidebar.



The cross-quarter comparison at **top_k=8** : Q1 FY2026 correctly identified as the highest at \$42,097M, each figure cited to its filing and page, all four quarters present in the sources.

INDEX STATUS

Indexed - 50 chunks ready

INDEXED FILINGS

Apple_Q1_FY2026_Earnings_Press_Release.pdf
12 chunks

Apple_Q2_FY2026_Earnings_Press_Release.pdf
12 chunks

Apple_Q3_FY2026_Earnings_Press_Release.pdf
11 chunks

Apple_Q4_FY2025_Earnings_Press_Release.pdf
15 chunks

CONFIGURATION

Embeddings: openai/text-embedding-3-small

Answering: openai/gpt-4o

Chunk size: 1200

Overlap: 200

Temperature: 0.0

RETRIEVAL

Chunks to retrieve (top_k): 8

Show retrieved chunks

Store: D:\Downloads\rag-submission\handoff\Finance-rag\chroma_db

RETRIEVAL AUGMENTED GENERATION

Quarterly Results Research Assistant

Ask about revenue, margins, dividends or management commentary. Every answer is drawn only from the indexed filings and carries the document and page it came from, so an analyst can verify it before it reaches a client.

CHUNKS INDEXED
50
1200 / 200 overlap

FILINGS
4
quarterly press releases

RETRIEVAL
top_k 8
cosine similarity

ANSWERING
GPT-4o
temperature 0

Upload and index documents

Ask a question

Sample questions from the assignment (optional)

Question

What is the CEO's personal shareholding in 2015?

Ask

ANSWER

What is the CEO's personal shareholding in 2015?

That information is not available in the uploaded documents.

SOURCES

Apple_Q2_FY2026_Earnings_Press_Release.pdf

p. 4 0.4375

p. 3 0.3985

Apple_Q3_FY2026_Earnings_Press_Release.pdf

p. 4 0.4370

p. 3 0.3903

Apple_Q1_FY2026_Earnings_Press_Release.pdf

The trap question. Nothing in these filings covers a personal shareholding in 2015, and the system says so rather than inventing a figure.

Quarterly Results RAG API 1.0.0 QAS 3.1

/openapi.json

Retrieval augmented generation over quarterly earnings press releases filed with the SEC.

default

POST /ingest Index one or more PDFs

POST /ask Ask a grounded question

GET /stats Collection statistics

Schemas

AskRequest > Expand all object

AskResponse > Expand all object

Body_ingest_endpoint_ingest_post > Expand all object

HTTPValidationError > Expand all object

IngestResponse > Expand all object

Source > Expand all object

StatsResponse > Expand all object

ValidationError > Expand all object

The FastAPI backend: `POST /ingest`, `POST /ask` and `GET /stats`, all live on the generated documentation page.

7. Results

The ten assignment questions, with the hand-verified answer key read directly out of the filings. This key is what the system's output is checked against, and the answers the system actually produced are in

`docs/test_answers.md` in the repository, with sources, similarity scores and retrieval spread for each.

Q1 — Revenue, most recent quarter

\$109.4 billion in Q3 FY2026 (quarter ended 27 June 2026), up 16% year over year. Total net sales of **\$109,417 million** in the statements.

Q2 — Net profit compared across quarters

Quarter	Net income
Q4 FY2025	\$27,466 M
Q1 FY2026	\$42,097 M ← highest
Q2 FY2026	\$29,578 M
Q3 FY2026	\$29,789 M

Q1 FY2026 is highest, which is the expected seasonal pattern — the December quarter carries the holiday iPhone cycle.

Q3 — Year on year

Q3 FY2026 revenue of **\$109,417 M** against **\$94,036 M** in the June 2025 quarter, **up 16%**. Net income **\$29,789 M** against **\$23,434 M**. Diluted EPS **\$2.02**, up 29%.

Q4 — Management commentary

Tim Cook (CEO) on Q3 FY2026: Apple's strongest June quarter ever, double-digit revenue growth across iPhone, Mac and Services and in every geographic segment, alongside the Siri AI launch at WWDC26. Kevan Parekh (CFO): new June-quarter records for EPS and operating cash flow, with the active installed base at an all-time high.

Q5 — Fastest-growing segment

Segment	Q3 FY26	Q3 FY25	Growth
Greater China	\$18,816 M	\$15,369 M	+22.43%
Europe	\$29,395 M	\$24,014 M	+22.41%
Rest of Asia Pacific	\$8,871 M	\$7,673 M	+15.6%
Japan	\$6,554 M	\$5,782 M	+13.4%
Americas	\$45,781 M	\$41,198 M	+11.1%

Greater China, by two hundredths of a percentage point. They are effectively tied, and a good answer says so rather than declaring a clean winner.

Q6 — Operating margin trend

The weakest answer in the system, and honestly flagged as such. See section 8.

Q7 — Dividend

Quarter	Per share	Record date	Payable
Q4 FY2025	\$0.26	10 Nov 2025	13 Nov 2025
Q1 FY2026	\$0.26	9 Feb 2026	12 Feb 2026
Q2 FY2026	\$0.27 (+4%)	11 May 2026	14 May 2026
Q3 FY2026	\$0.27	10 Aug 2026	13 Aug 2026

Q8 — Risks and headwinds

Thinner than the other answers, and legitimately so. The releases carry the standard forward-looking-statements caveat, and Q3 FY2026 discloses roughly 2 percentage points of favourable gross-margin impact from **tariff refunds**, with \$0.11 of EPS attributable to the same — a real, quantified, non-recurring item. A press release is not a risk-factors section; the 10-Q is the right document for a full risk answer.

Q9 — Three-line client summary

A synthesis question with no single right answer. It should draw on Q3 FY2026 revenue of \$109.4 billion (+16%), EPS of \$2.02 (+29%), the June-quarter records, and the tariff-refund caveat on margin.

Q10 — Trap question

"What is the CEO's personal shareholding in 2015?"

Nothing in the four documents touches Tim Cook's personal shareholding, and none of them covers 2015. The system must reply that the information is not available in the uploaded documents. **It must not invent a figure.**

8. What does not work well

Operating margin (Q6) is the weakest answer, and it is an arithmetic problem rather than a retrieval one.

The press releases headline *gross margin* (50.1% in Q3 FY2026). Operating margin must be derived from operating income divided by net sales, for each of four quarters — twelve numbers across four documents, then a trend judgement. GPT-4o will sometimes report gross margin and label it operating margin, which is exactly the confidently-wrong output an analyst would act on. Retrieval is fine; the chunks contain the right rows. A calculation tool would fix this properly and is outside what the brief permits.

Multi-column table extraction is lossy. Apple's statements place the current quarter, the prior-year quarter and the year-to-date columns side by side. `pypdf` flattens these into a run of numbers, and the model can read a year-to-date figure as a quarterly one. The \$101,464 M nine-month net income sitting on the same row as the \$29,789 M quarterly figure is a live trap. Anyone using this for real work must click through to the citation.

The source PDFs were rendered from EDGAR HTML. SEC filings are published as HTML, so Exhibit 99.1 was rendered to PDF. The text layer is clean and the content unmodified, but the page breaks are the renderer's, so cited page numbers correspond to this repository's PDFs rather than to any Apple pagination.

50 chunks is a small corpus. With `top_k=8`, retrieval touches over a sixth of the collection on every query, so retrieval quality flatters itself here. On a realistic corpus this configuration would need re-tuning.

9. Running it

```
git clone https://github.com/Harshit-S/finance-rag
cd finance-rag

python -m venv .venv
.venv\Scripts\activate           # Windows; source .venv/bin/activate elsewhere
pip install -r requirements.txt

cp .env.example .env             # then paste your OPENAI_API_KEY into it

python ingest.py                  # -> "4 files processed, 50 chunks stored."
streamlit run app.py              # http://localhost:8501
uvicorn api.main:app --reload     # http://localhost:8000/docs
```

API endpoints (section 6 bonus)

Method	Endpoint	Input	Output
POST	/ingest	One or more PDF files	{"files": 4, "chunks": 50, ...}
POST	/ask	{"question": "...", "top_k": 8}	{"answer": "...", "sources": [...]}
GET	/stats	—	Collection, chunk count, models

Repository layout

```
finance-rag/
├── app.py                # Streamlit interface
├── ui.py                 # stylesheet and HTML fragments
├── ingest.py             # load, chunk, embed, store in Chroma
├── rag.py                # retrieve + prompt + call GPT-4o
├── config.py             # all tunables in one place
├── run_test_questions.py # runs the 10 assignment questions
├── api/main.py           # FastAPI backend (bonus)
├── data/                 # the four quarterly PDFs
├── chroma_db/            # persisted vector store (gitignored)
├── docs/                 # answers + screenshots (see docs/README.md)
├── deploy/              # Render blueprint, Dockerfile, hosting notes
└── requirements.txt
```